

Proposal

Degree Program Computer Science BIF

Course Scientific Writing

Barrierefreiheit von DevOps-Weboberflächen: GitHub, GitLab und Azure DevOps für seh- und motorisch eingeschränkte Entwickler:innen

By: Ron Feldmann

Student number: 2210257080

Advisor: Mag. Dr. Christian Osterbauer

Wien, 16.01.2026

Contents

1	Einführung	4
1.1	Motivation	4
1.2	Zentrale Forschungsfrage	4
1.3	Unterfragen.....	5
2	Aktueller Stand der Forschung	5
3	Relevanz.....	6
4	Methoden.....	7
4.1	Literatur- und Dokumentenanalyse.....	7
4.2	Ableitung von Nutzungsszenarien	7
4.3	Heuristische Evaluation.....	7
4.3.1	Tastaturtests (Keyboard-only).....	8
4.3.2	Screenreader-Tests (NVDA)	8
4.3.3	Ablauf und Dokumentation der Evaluation	8
4.4	Entwurf und Implementierung einer Browser-Erweiterung.....	9
4.5	Szenariobasierte Nutzerstudie	9
5	Erwartete Ergebnisse	10
6	Vorläufige Ergebnisse	11
6.1	GitHub – Tastaturtests (Keyboard-only)	11
6.1.1	Workflow „Issues“	11
6.1.2	Workflow „Pull-/Merge-Requests“	13
6.2	Github – Screenreader-Tests (NVDA)	14
6.3	GitLab – Tastaturtests (Keyboard-only).....	15
6.3.1	Workflow „Issues/Work items“ (Listenansicht).....	15
6.3.2	Workflow „Merge Requests“	16

6.4	GitLab – Screenreader-Tests (NVDA)	16
6.4.1	Work Items / Boards	16
6.4.2	Pull Requests	16
6.5	Azure DevOps – Tastaturtests (Keyboard-only)	16
6.5.1	Work Items / Boards	16
6.5.2	Pull Requests	16
6.6	Azure DevOps – Screenreader (NVDA)	16
6.6.1	Work Items / Boards	16
6.6.2	Pull Requests	16
7	Diskussion	17
8	Zeitplan	18
9	Abbildungsverzeichnis	18
10	Quellenverzeichnis	Error! Bookmark not defined.

1 Einführung

1.1 Motivation

Webbasierte DevOps-Plattformen wie GitHub, GitLab oder Azure DevOps gehören heute zum festen Bestandteil vieler Softwareprojekte. Neben der reinen Versionsverwaltung unterstützen sie Entwicklerteams dabei, Arbeit zu planen, Code-Reviews durchzuführen und Build- sowie Deployment-Pipelines zu überwachen. Ein großer Teil der Abstimmung von Aufgaben und Ergebnissen findet damit direkt in den Weboberflächen dieser Plattformen statt.

Dass eine Oberfläche im Browser läuft, bedeutet jedoch nicht automatisch, dass sie für alle gleichermaßen gut bedienbar ist. Für Programmierer:innen, die zum Beispiel mit Screenreader, Tastatur oder anderen Eingabegeräten arbeiten, können scheinbar kleine Details – etwa eine schlecht sichtbare Fokusbildung, eine wenig klar strukturierte Seite oder Funktionen, die nur mit der Maus erreichbar sind – schnell zu spürbaren Hürden werden. Als fachliche Referenz für die Beurteilung von Web-Anwendungen dienen in dieser Arbeit die Web Content Accessibility Guidelines (WCAG) in Version 2.2, die unter anderem Anforderungen an Tastaturnebenbarkeit, Fokusbildung und Zielgrößen enthalten [1].

Das Ziel dieser Bachelorarbeit besteht darin, die Barrierefreiheit ausgewählter DevOps-Workflows in den Weboberflächen von GitHub, GitLab und Azure DevOps für seh- und motorisch eingeschränkte Entwickler:innen zu untersuchen. Im Fokus stehen die folgenden DevOps-Workflows:

- Bearbeiten von Issues
- Erstellen und Reviewen von Pull-/Merge-Requests
- Überwachen von CI/CD-Pipelines (ergänzend)

1.2 Zentrale Forschungsfrage

„Wie barrierefrei sind ausgewählte DevOps-Workflows in den Weboberflächen von GitHub, GitLab und Azure DevOps für seh- und motorisch eingeschränkte Entwickler:innen gemäß den Anforderungen der WCAG 2.2?“

1.3 Unterfragen

Zur Konkretisierung der Forschungsfrage werden folgende Unterfragen betrachtet:

1. Welche WCAG-2.2-Erfolgskriterien (Level A/AA) sind für DevOps-Weboberflächen besonders relevant?
2. Welche Barrieren treten in ausgewählten DevOps-Workflows der drei Plattformen, insbesondere bei der Nutzung mit Tastatur und Screenreader, auf?
3. In welchen Bereichen unterscheiden sich GitHub, GitLab und Azure DevOps hinsichtlich der Barrierefreiheit?
4. Welche wiederkehrenden Accessibility-Probleme lassen sich identifizieren?

Die Arbeit konzentriert sich auf Entwickler:innen mit Seh- und motorischen Beeinträchtigungen und verwendet ausgewählte Erfolgskriterien der WCAG 2.2 auf Level A und AA als Bewertungsrahmen.

2 Aktueller Stand der Forschung

Es gibt bereits einige Arbeiten zu Barrieren in Entwicklungswerkzeugen für blinde und sehbehinderte Entwickler:innen. Mealin und Murphy-Hill beschreiben typische Schwierigkeiten blinder Softwareentwickler:innen beim Einsatz gängiger Entwicklungstools [2]. Potluri et al. stellen mit CodeTalk ein Plugin für Visual Studio vor, das die Navigation und Informationsdarstellung für sehbehinderte Entwickler:innen verbessert und in ersten Studien positiv bewertet wurde [3]. Pandey et al. untersuchen die Barrierefreiheit gängiger UI-Frameworks und Bibliotheken und zeigen deren Einfluss auf die Arbeit von Entwickler:innen mit Sehbeeinträchtigungen auf [4]. Nowrin und Vertanen analysieren mit „Programming by Voice“, inwiefern sich Sprachsteuerung als alternative Eingabemethode für das Programmieren eignet [5].

Zusätzlich gibt es neuere Arbeiten, die zeigen, dass gerade in komplexen Webanwendungen wiederkehrende Barrieren bei Tastaturbedienung und Fokusmanagement auftreten. Solche Probleme sind für DevOps-Plattformen besonders relevant, da die Weboberflächen stark interaktiv sind und zentrale Aufgaben (z. B. Reviews oder Pipeline-Ansichten) aus vielen dynamischen UI-Elementen bestehen [6].

Auch im Umfeld von GitHub werden Accessibility-Probleme in der Praxis regelmäßig sichtbar: Untersuchungen zu populären GitHub-Projekten zeigen wiederkehrende Muster von Accessibility-Issues und machen deutlich, dass Barrierefreiheit im Entwicklungsalltag ein reales Thema ist [7].

Diese Arbeiten fokussieren sich jedoch vorwiegend auf IDEs, UI-Frameworks oder spezielle Assistenzsysteme. Die webbasierten Oberflächen großer DevOps-Plattformen wurden bislang kaum systematisch aus der Perspektive seh- oder motorisch eingeschränkter Entwickler:innen untersucht, insbesondere nicht im Hinblick auf konkrete DevOps-Workflows und die neuen Erfolgskriterien von WCAG 2.2 [1] sowie vor dem Hintergrund der aktuellen EU-Richtlinien zur digitalen Barrierefreiheit [8], [9]. Hier setzt die geplante Arbeit an.

3 Relevanz

DevOps-Plattformen sind in vielen Teams das zentrale Werkzeug für die Zusammenarbeit rund um den Code. Sind zentrale Workflows in diesen Benutzeroberflächen nicht barrierefrei, schränkt das die gleichberechtigte Teilhabe von Entwickler:innen mit Seh- oder Motorikbeeinträchtigungen direkt ein.

Gleichzeitig schaffen die EU-Richtlinie 2016/2102 und der European Accessibility Act klare Erwartungen an digitale Barrierefreiheit [8], [9].

Im europäischen Kontext spielt außerdem EN 301 549 als verbreiteter Anforderungskatalog für digitale Barrierefreiheit eine wichtige Rolle, wodurch die Ergebnisse der Arbeit auch für den öffentlichen Bereich und Compliance-Fragen besser einzuordnen sind [10].

Die Arbeit adressiert somit ein reales, praktisches Problem: Wie können bestehende Plattformen so verbessert werden, dass sie auch mit Tastatur, Screenreader oder alternativen Eingabegeräten effizient nutzbar sind? Erste Kontakte zur bzw. zum Barrierefreiheitsbeauftragten des Parlaments bestehen bereits, was die spätere Rekrutierung von Proband:innen realistischer macht.

4 Methoden

4.1 Literatur- und Dokumentenanalyse

Zunächst erfolgt eine Recherche zu WCAG 2.2 (Level A/AA) sowie zu bestehenden Arbeiten über die Bedürfnisse seh- und motorisch eingeschränkter Entwickler:innen [1], [2], [3], [4], [5]. Zusätzlich werden die EU-Richtlinien 2016/2102 und 2019/882 sowie die offiziellen Accessibility-Dokumentationen von GitHub, GitLab und Azure DevOps ausgewertet [8], [9].

Ergänzend werden vorhandene Accessibility Conformance Reports (z. B. VPAT/ACR) der Plattformen herangezogen, um die eigenen Beobachtungen aus der Evaluation einzuordnen [11], [12], [13].

4.2 Ableitung von Nutzungsszenarien

Auf Basis der Literatur sowie einer ersten Exploration der Oberflächen werden zwei bis drei zentrale Nutzungsszenarien definiert:

- Bearbeiten von Issues
- Erstellen und Reviewen von Pull-/Merge-Requests
- Fehlgeschlagene CI/CD-Pipeline finden (ergänzend)

Die Szenarien werden den relevanten WCAG-2.2-Erfolgskriterien der Level A/AA zugeordnet [1].

4.3 Heuristische Evaluation

Die Weboberflächen der drei Plattformen werden in einer szenariobasierten heuristischen Evaluation mit Tastaturnavigation und Screenreader untersucht. Das Vorgehen orientiert sich dabei an etablierten Empfehlungen zur WCAG-basierten Evaluation (z. B. WCAG-EM), wird hier jedoch bewusst szenariobasiert auf konkrete DevOps-Workflows angewendet [14].

Gefundene Barrieren werden nach Schweregrad, nach der betroffenen Nutzergruppe (z. B. Screenreader-Nutzer:innen, reine Tastaturnutzende) und nach der betroffenen Funktion klassifiziert.

Die Evaluation orientiert sich an den in Abschnitt 4.2 definierten Nutzungsszenarien und bildet die Grundlage zur Beantwortung der Unterfragen zu konkreten Barrieren und Unterschieden zwischen den Plattformen. Positive Beispiele werden ebenfalls dokumentiert. GitHub wurde bereits evaluiert; GitLab/Azure folgen.

4.3.1 Tastaturtests (Keyboard-only)

Die Tastaturtests wurden im Browser **Google Chrome** durchgeführt. Zur Navigation und Interaktion wurden **Tab/Shift+Tab** (Fokuswechsel), **Enter/Space** (Aktivierung) sowie **Pfeiltasten** (Arrow Keys) verwendet. Zusätzlich wurde beobachtet, ob die Navigation konsistent bleibt (z. B. nach Seitenwechseln, Filterwechseln oder Mausinteraktion), und ob sichtbare Bedienelemente zuverlässig erreichbar sind. Besonderes Augenmerk lag auf Fokusreihenfolge, Fokusverhalten und der Erreichbarkeit zentraler Funktionen ohne Maus.

4.3.2 Screenreader-Tests (NVDA)

Für die Screenreader-Perspektive wird **NVDA** unter **Windows** in Kombination mit **Google Chrome** verwendet. Fokus der Screenreader-Tests ist, ob Struktur und Bedienelemente verständlich angekündigt werden (z. B. zugängliche Namen von Buttons, sinnvolle Struktur/Beziehungen, konsistente Interaktion mit Fokus).

4.3.3 Ablauf und Dokumentation der Evaluation

Die Evaluation wird pro Plattform anhand der in Abschnitt 4.2 definierten Nutzungsszenarien durchgeführt. Für jedes Szenario wird ein reproduzierbarer Startpunkt festgelegt (z. B. Repository-Issues-Übersicht oder konkretes Issue bzw. Pull/Merge Request). Die Aufgaben werden anschließend schrittweise durchgeführt und dabei Fokusverhalten, Erreichbarkeit von Bedienelementen sowie Konsistenz der Navigation beobachtet.

Alle Befunde werden in einem Barrierenlog dokumentiert. Pro Eintrag werden Plattform, Workflow-Schritt, beobachtetes Problem, betroffene Nutzergruppe, zugeordnete WCAG-Erfolgskriterien (Level A/AA), Schweregrad sowie kurze Reproduktionshinweise festgehalten. Je nach Fall werden ergänzend Screenshots und DOM-Hinweise (z. B. tabindex, role, aria-Attribute) notiert, um die Beobachtungen nachvollziehbar zu machen.

Bei unklaren Fällen werden Beobachtungen wiederholt und nach Zustandswechseln (z. B. Open/Closed, Filterwechsel) erneut geprüft, um zustandsabhängiges Verhalten zu erkennen.

4.4 Entwurf und Implementierung einer Browser-Erweiterung

Aus den Evaluationsergebnissen wird für eine Plattform (voraussichtlich GitHub) eine Prioritätenliste zentraler Barrieren erstellt. Auf dieser Grundlage wird eine Browser-Erweiterung (WebExtension) entwickelt, die ausgewählte Probleme adressiert, zum Beispiel durch:

- verbesserte Fokusdarstellung,
- zusätzliche Tastaturkürzel,
- ergänzte ARIA-Attribute,
- tastaturbedienbare Alternativen zu Drag-and-Drop.

Für Anpassungen an ARIA und Keyboard-Interaktionen dienen WAI-ARIA sowie die ARIA Authoring Practices als technische Referenz, um Änderungen möglichst kompatibel zu gestalten [15], [16].

Der Prototyp konzentriert sich auf die wichtigsten Barrieren in ein bis zwei der zuvor definierten Workflows (z. B. Bearbeitung von Issues und Pull-/Merge-Requests) auf GitHub und soll exemplarisch zeigen, wie wiederkehrende Accessibility-Probleme auf der Clientseite entschärft werden können.

4.5 Szenariobasierte Nutzerstudie

In einer kleinen Nutzerstudie (ca. sechs bis zehn Personen, möglichst inklusive Entwickler:innen mit Seh- oder Motorikbeeinträchtigungen) werden ausgewählte Szenarien – voraussichtlich ein bis zwei zentrale Workflows wie die Bearbeitung von Issues und Pull-/Merge-Requests – einmal ohne und einmal mit aktivierter Erweiterung bearbeitet. Erfasst werden unter anderem:

- Bearbeitungszeit,
- Erfolgsquote,
- subjektive Zufriedenheit (z. B. per SUS-Fragebogen).

Die Auswertung erfolgt primär deskriptiv. Bei ausreichender Datenlage werden einfache gepaarte Tests eingesetzt.

5 Erwartete Ergebnisse

Die Arbeit soll ein klareres Bild davon liefern, wie gut die ausgewählten DevOps-Workflows auf GitHub, GitLab und Azure DevOps derzeit für seh- und motorisch eingeschränkte Entwickler:innen nutzbar sind. Auf Basis der geplanten Analyse und Evaluation sind insbesondere folgende Ergebnisse zu erwarten:

- Ein strukturierter Überblick über Barrieren und positive Beispiele in den betrachteten Workflows, geordnet nach Plattform, Workflow-Schritt und den jeweils betroffenen WCAG-2.2-Erfolgskriterien (Level A/AA).
- Ein Vergleich der drei Plattformen, aus dem hervorgeht, in welchen Bereichen sich GitHub, GitLab und Azure DevOps hinsichtlich der Barrierefreiheit unterscheiden und welche wiederkehrenden Accessibility-Probleme plattformübergreifend auftreten.
- Ein lauffähiger Prototyp einer Browser-Erweiterung für eine Plattform (voraussichtlich GitHub), der ausgewählte Barrieren in ein bis zwei Workflows adressiert, zum Beispiel durch verbesserte Tastaturbedienbarkeit, Fokusbildung und semantische Auszeichnung.
- Erste empirische Hinweise aus einer kleinen Nutzerstudie darauf, ob sich die ausgewählten Aufgaben mit der Erweiterung schneller, zuverlässiger oder subjektiv angenehmer bearbeiten lassen als ohne.
- Konkrete Empfehlungen für Anbieter von DevOps-Plattformen und Entwicklungsteams, worauf bei der Gestaltung barrierearmer DevOps-Weboberflächen und bei der Unterstützung seh- und motorisch eingeschränkter Entwickler:innen besonders zu achten ist.

6 Vorläufige Ergebnisse

Die folgenden vorläufigen Ergebnisse basieren auf der szenariobasierten Evaluation mit **Keyboard-only in Google Chrome**. Befunde wurden als Barrieren dokumentiert und nach Schweregrad eingeordnet. Die Einstufung orientiert sich daran, ob ein Workflow-Schritt zuverlässig ausführbar ist, wie viel Aufwand bzw. Umwege nötig sind und ob eine Barriere voraussichtlich viele Nutzer:innen betrifft.

- **S1 (kritisch):** Der Workflow-Schritt ist mit der jeweiligen Methode nicht zuverlässig oder praktisch gar nicht durchführbar. Betroffene Nutzer:innen bleiben an dieser Stelle hängen oder müssen den Schritt abbrechen (Blocker).
- **S2 (hoch):** Der Workflow-Schritt ist grundsätzlich möglich, aber stark eingeschränkt, z. B. nur über Umwege oder mit deutlich erhöhtem Aufwand. Typisch sind Fehlbedienungen, Orientierungsschwierigkeiten oder eine stark reduzierte Effizienz.
- **S3 (mittel):** Der Workflow-Schritt ist grundsätzlich machbar, aber umständlich oder inkonsistent. Workarounds sind möglich, dennoch entstehen unnötige Zusatzschritte oder wiederkehrende Irritationen.
- **S4 (Beobachtung/positiv):** Es liegt kein klarer Fehler vor, sondern ein korrekt umgesetztes Interaktionsmuster oder ein erwartbares, kontextabhängiges Verhalten. Diese Punkte werden als positives Beispiel oder als Hinweis für die Interpretation dokumentiert.

Die Zuordnung der Befunde zu WCAG 2.2 erfolgt anhand der Erfolgskriterien [1].

6.1 GitHub – Tastaturtests (Keyboard-only)

6.1.1 Workflow „Issues“

In GitHub zeigen sich in der Issues-Ansicht mehrere Probleme, die sich vor allem um **Fokusmanagement** und **Tastaturzugänglichkeit** drehen. Besonders auffällig ist ein zustandsabhängiges Verhalten: Je nachdem, ob eine Seite frisch geladen wurde, ob zwischen „Open“ und „Closed“ gewechselt wurde oder ob zuvor eine Mausinteraktion stattgefunden hat, verändert sich die Tastaturnavigation.

(S1) Issues-Liste: inkonsistente Tastaturnavigation (roving tabindex)

In der Issues-Liste ist jeweils nur ein Eintrag direkt per Tab erreichbar, während andere Einträge aus der Tab-Reihenfolge entfernt sind (roving tabindex; ein Eintrag mit tabindex="0", alle anderen mit tabindex="-1"). In der Praxis ist die Navigation jedoch nicht zuverlässig: Einträge werden teilweise übersprungen oder der Fokus springt unerwartet an eine andere Stelle (z. B. in Richtung Footer). Dadurch ist die Liste nicht reproduzierbar

konsistent bedienbar (siehe Abbildung 1).

WCAG-Bezug: 2.1.1 Keyboard, 2.4.3 Focus Order.

Schweregrad: S1.

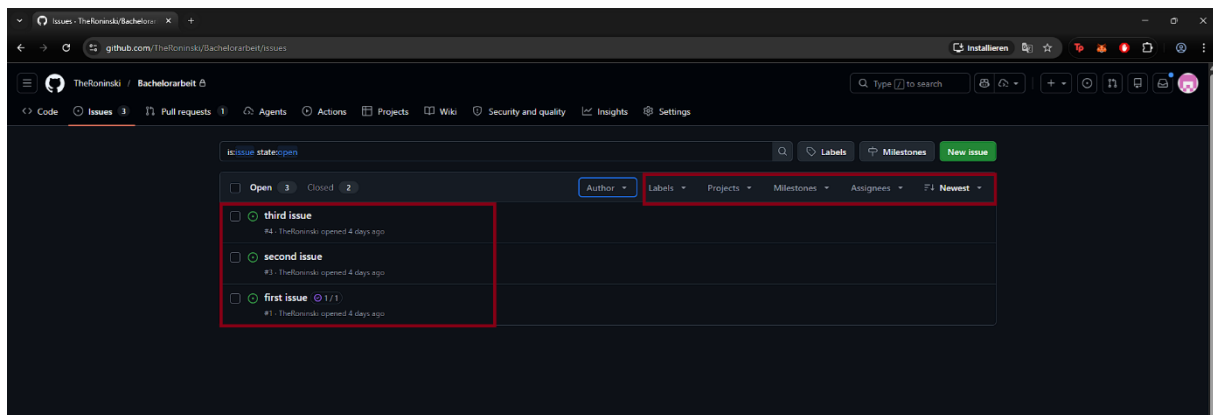


Abbildung 1: Github Issues-Übersicht. Markiert sind die Issues-Liste (links) und die Filter-Toolbar (rechts), in denen Barrieren bei Tab-/Fokus-Navigation beobachtet wurden.

(S1) Filter-Toolbar: zentrale Filter nicht per Tab erreichbar

In der Filter-Toolbar ist nur das erste Element per Tab erreichbar. Weitere zentrale Filter (z. B. Labels, Projects, Milestones, Assignees oder Sortierung) sind aus der Tab-Reihenfolge entfernt. In manchen Fällen verlässt Tab die Toolbar komplett, bevor alle sichtbaren Elemente erreicht werden konnten (siehe Abbildung 1).

WCAG-Bezug: 2.1.1, 2.4.3.

Schweregrad: S1.

(S1) Fokusverhalten abhängig von Interaktion (nicht vorhersehbar)

Nach Mausinteraktion (z. B. Klick auf einen Issue) verändert sich das Tab-Verhalten. Der Fokus bleibt an „aktiven“ Elementen hängen oder verhält sich nicht mehr konsistent, wodurch die Bedienung schwer vorhersehbar wird.

WCAG-Bezug: 2.4.3 Focus Order.

Schweregrad: S1.

(S1) Markdown-Toolbar: sichtbare Buttons nicht tabbbar

Im Kommentar-/Editorbereich sind Formatierungsbuttons sichtbar (z. B. Überschrift, Fett, Kursiv, Listen), aber nicht per Tab erreichbar (tabindex="-1"). Damit sind zentrale Editorfunktionen nicht direkt über die Tastatur erreichbar, sondern nur über Umwege (siehe Abbildung 2).

WCAG-Bezug: 2.1.1, 2.4.3.

Schweregrad: S1.

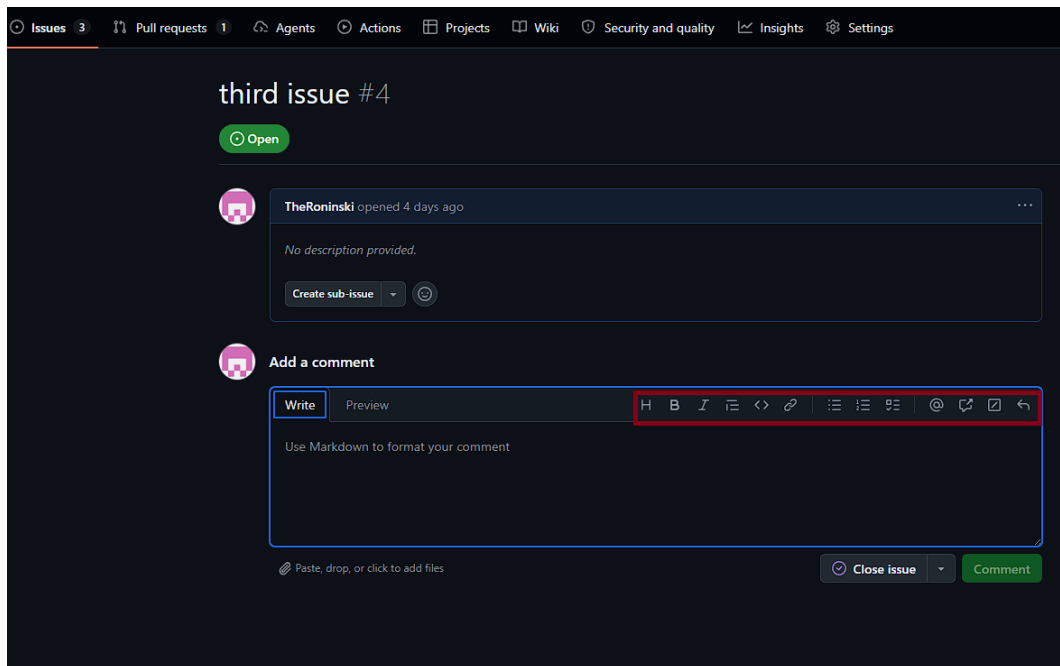


Abbildung 2: GitHub Issue-Detailansicht. Markiert ist die Markdown-Toolbar im Kommentar-Editor, deren Buttons sichtbar sind, aber nicht direkt per Tab erreichbar sind.

(S3) Interaktionskonsistenz: Wechsel zwischen Tab und Pfeiltasten

In verschiedenen Bereichen werden unterschiedliche Interaktionsmodelle verwendet. Teilweise ist Tab-Navigation naheliegend, in anderen Bereichen ist Arrow-Key-Navigation notwendig. Das ist nicht grundsätzlich falsch, erhöht aber den Aufwand und kann in Kombination mit instabilem Fokusverhalten problematisch werden.

Schweregrad: S3.

6.1.2 Workflow „Pull-/Merge-Requests“

Auch im PR-Kontext treten ähnliche Muster wie bei Issues auf, insbesondere bei Toolbars und Aktionsbereichen. Gleichzeitig gibt es auch Beispiele für korrekt umgesetzte ARIA-Patterns.

(S1/S3) Toolbars und Action-Bars: wiederkehrende Tab-/Fokus-Probleme

In mehreren Bereichen sind sichtbare Buttons aus der Tab-Reihenfolge entfernt oder der Fokus bleibt an einzelnen Elementen hängen. Je nach Stelle kann dies zu Blockern (S1) oder „nur“ zu Umständlichkeit/Inkonsistenz (S3) führen.

Schweregrad: S1–S3.

(S4) PR-Header Tabs (Conversation/Commits/Files...): korrektes Pattern

Die Tab-Navigation folgt einem Tablist-Pattern (role="tablist", role="tab") mit roving tabindex.

Die Navigation mit Pfeiltasten ist erwartbar und grundsätzlich korrekt umgesetzt.

Schweregrad: S4 (kein Fehler; positives Beispiel).

(S4) Dropdown-Menüs: grundsätzlich erwartbares Verhalten

In Dropdown-Menüs erfolgt Navigation typischerweise mit Pfeiltasten, Tab verlässt das Menü. Dieses Verhalten entspricht grundsätzlich dem erwartbaren ARIA-Menüverhalten, sofern Fokus korrekt gesetzt bleibt.

Schweregrad: S4.

6.2 Github – Screenreader-Tests (NVDA)

In einem ersten NVDA-Test (Windows) des Issues-Workflows in Google Chrome wurden in den überprüften Bereichen keine eindeutigen Stellen gefunden, die speziell für Screenreader-Nutzer:innen eine klare Barriere darstellen. Grundlegende Bereiche wie Navigation, Issues-Liste, Filterbereich und die Detailansicht eines Issues wurden insgesamt verständlich angekündigt. Auch interaktive Elemente wie Links und Menüs hatten in den meisten Fällen sinnvolle zugängliche Namen und Rollen.

So wurde beispielsweise der Link „Milestones“ von NVDA mit einem klaren Namen und der erwarteten Rolle („link“) angekündigt. Ebenso wurde das Reactions-Menü in der Issue-Detailansicht als Menü bzw. Submenü erkannt und entsprechend angesagt. Das spricht dafür, dass hier grundsätzlich mit einem korrekt umgesetzten Bedienkonzept gearbeitet wird. Diese Stellen werden als positive Beispiele dokumentiert (S4).

Gleichzeitig fiel auf, dass die Menge der angesagten Informationen je nach Element stark variieren kann. Besonders bei zusammengesetzten Filterausdrücken im Suchfeld oder bei Elementen mit zusätzlichem Kontext (z. B. Autor:in und Zeitstempel) kann die Ausgabe relativ ausführlich werden. Das ist funktional grundsätzlich korrekt und kann auch hilfreich sein, kann bei häufiger Navigation aber die Orientierung und Effizienz beeinträchtigen. Diese Punkte werden daher als Beobachtung zur Informationsdichte eingeordnet (S3), ohne dass im aktuellen Stand ein eindeutiger WCAG-Verstoß daraus abgeleitet wird.

Schweregrad (Kurzfassung): überwiegend S4 (korrekt/positiv), vereinzelt S3-Beobachtungen zur Informationsdichte.

Hinweis: Die Screenreader-Perspektive wird im weiteren Verlauf nach demselben Schema auch auf GitLab (und bei Bedarf Azure DevOps) übertragen, um Unterschiede zwischen den Plattformen besser vergleichen zu können.

6.3 GitLab – Tastaturtests (Keyboard-only)

6.3.1 Workflow „Issues/Work items“ (Listenansicht)

Im GitLab-Workflow „Work items“ zeigt sich insgesamt eine vergleichsweise stabile Tastaturnavigation. Die Fokusreihenfolge ist im Allgemeinen nachvollziehbar: Beginnend bei Elementen der Top-Navigation (z. B. globale Suche, „Create“-Menü/Plus-Button, Profilmenu) lässt sich anschließend die linke Projekt-Sidebar (Navigation) mit Tab schrittweise durchlaufen. Auch das Hauptpanel („Work items“-Seite) ist grundsätzlich erreichbar und die Navigation setzt dort erwartbar fort.

(S3) Filter-/Suchbereich: Navigation erfordert Pfeiltasten (kombiniertes Interaktionsmodell)

Im Filter-/Suchbereich der „Work items“-Ansicht („filtered search“) lässt sich der Fokus zwar per Tab in den Suchbereich setzen, jedoch kann man dort ohne aktive Auswahl bzw. ohne definierte Filter nicht im erwarteten Tab-Modus sinnvoll fortfahren. Stattdessen ist es notwendig, innerhalb des Filter-Widgets mit Pfeiltasten (Arrow Keys) durch Vorschläge/Tokens zu navigieren und Auswahlaktionen mit Enter auszuführen. Erst nachdem ein Filter/Token gewählt wurde, kann die Tab-Navigation wieder wie erwartet fortgesetzt werden.

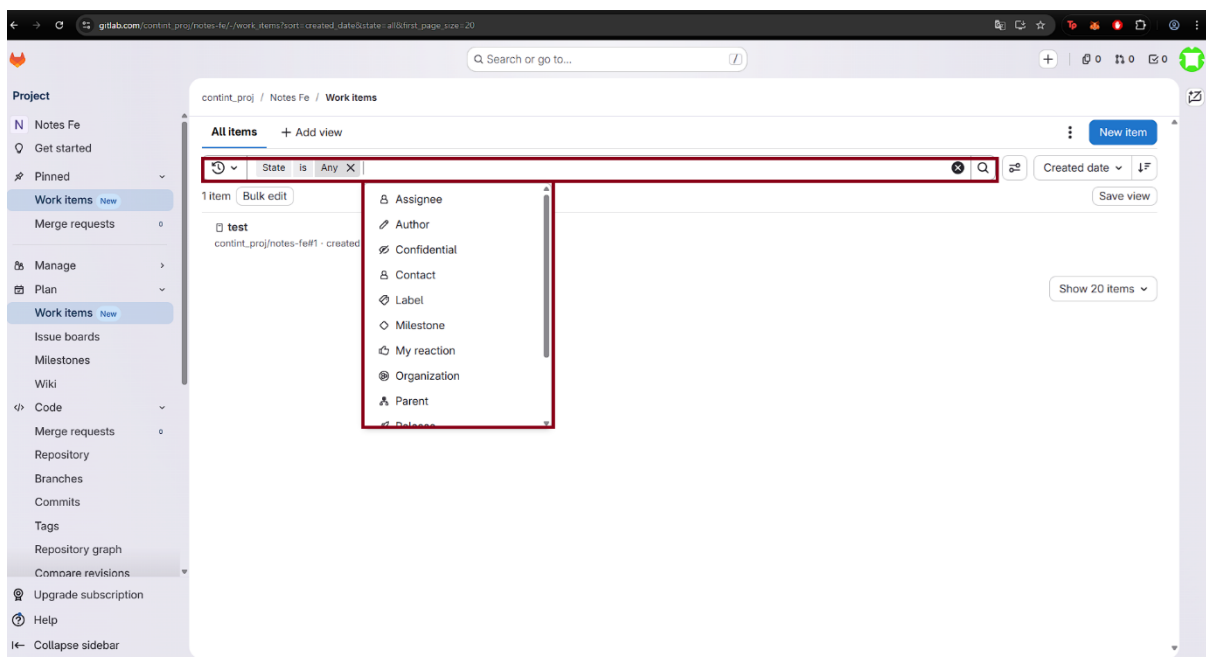


Abbildung 3: GitLab „Work items“ – Filter-/Suchbereich mit geöffnetem Token-Dropdown. Innerhalb des Widgets erfolgt die Auswahl typischerweise über Pfeiltasten und Enter, bevor die Tab-Navigation wieder fortgesetzt werden kann.

Diese Stelle ist kein vollständiger Blocker, da ein Workaround vorhanden ist (Arrow Keys + Enter). Für reine Tastaturnutzende kann der Wechsel des Interaktionsmodells jedoch irritierend sein, weil im restlichen Workflow primär Tab/Shift+Tab genutzt wird und nicht unmittelbar ersichtlich ist, dass innerhalb des Filter-Widgets ein „Listbox/Combobox“-ähnliches Bedienmuster aktiv ist.

WCAG-Bezug: 2.1.1 Keyboard (Bedienbarkeit grundsätzlich möglich, aber nicht konsistent im erwarteten Tab-Modell), 3.2.3 Consistent Navigation (als Interpretationshilfe; Interaktionsmodell wechselt innerhalb des Workflows).

Schweregrad: S3.

Kurzfazit GitLab (Keyboard-only):

Abgesehen vom Filter-/Suchbereich ist die Tastaturnavigation im getesteten Bereich stabil und nachvollziehbar. Die auffälligste Beobachtung ist die notwendige Umstellung auf Pfeiltasten im Filter-Widget, um dort weiterzuarbeiten.

6.3.2 Workflow „Merge Requests“

6.4 GitLab – Screenreader-Tests (NVDA)

6.4.1 Work Items / Boards

6.4.2 Pull Requests

6.5 Azure DevOps – Tastaturtests (Keyboard-only)

6.5.1 Work Items / Boards

6.5.2 Pull Requests

6.6 Azure DevOps – Screenreader (NVDA)

6.6.1 Work Items / Boards

6.6.2 Pull Requests

7 Diskussion

Bei den bisher bei GitHub durchgeführten Tastaturtests zeigt sich, dass Barrieren besonders dort auftreten, wo komplexe, sich ändernde UI-Komponenten mit Fokussteuerung zum Einsatz kommen. Insbesondere problematisch sind Stellen, an denen sichtbare Dinge nicht zuverlässig per Tab ansteuerbar sind bzw. sich das Fokusverhalten abhängig vom Zustand bzw. von vorausgegangenen Interaktionen ändert. In der Issues-Ansicht betrifft das vor allem die Issues-Liste sowie die Filter-Toolbar. Solche Probleme wirken sich direkt darauf aus, wie User:innen normal arbeiten, denn Navigation und Filtern sind grundlegende Schritte, damit wir schnell die Issues finden, die wir brauchen und bearbeiten.

Interessant ist zudem, dass GitHub an manchen Stellen moderne ARIA-Pattern mit einsetzt (z. B. Tablist im PR-Header), dort aber nicht immer einhält. Es scheint also nicht das Problem, dass GitHub solche Pattern verwendet, sondern, dass sie nicht immer überall eingesetzt bzw. installiert sind. Für die weitere Arbeit ist es also spannend, an welchen Stellen wir immer wieder ähnliche Probleme sehen (z. B. Nutzung von roving tabindex in Listen/Toolbars) und welche Stellen/Komponenten bereits heute als Vorbild gelten können.

Für QG2 sind die Ergebnisse bewusst noch vorläufig. Im Moment beziehen sie sich auf Keyboard-only Tests in Google Chrome. Die Screenreader-Perspektive mit NVDA wird als nächster Schritt ergänzt. Damit lässt sich besser einschätzen, wie stark sich die gefundenen Probleme auf Screenreader-Nutzer:innen auswirken, zum Beispiel wenn Buttons keinen klaren Namen haben oder wenn die Struktur einer Seite nicht sauber erkennbar ist.

Natürlich gibt es auch Einschränkungen. Weboberflächen ändern sich laufend, und damit kann sich auch das Verhalten von Fokus oder Navigation verändern. Außerdem ist „motorische Einschränkung“ sehr breit. Mit Keyboard-only kann man zwar einen wichtigen Teil abdecken, aber nicht jede reale Nutzung mit alternativen Eingabegeräten vollständig nachstellen. Und auch die geplante Nutzerstudie wird klein sein. Sie soll eher erste Hinweise liefern, ob der Prototyp in der Praxis tatsächlich hilft, und ist nicht dafür gedacht, repräsentative Aussagen zu machen.

Als nächstes folgt derselbe Ablauf für GitLab und, wenn es zeitlich sinnvoll bleibt, auch Azure DevOps. Ziel ist, die Ergebnisse im gleichen Schema zu dokumentieren, damit sich Gemeinsamkeiten und Unterschiede sauber vergleichen lassen. Parallel dazu starte ich die NVDA-Tests in den gleichen Workflows. Auf Basis der priorisierten Barrieren entsteht dann der Prototyp als Browser-Erweiterung für GitHub, zunächst bewusst fokussiert auf ein bis zwei Workflows. Wenn der Prototyp stabil ist, führe ich einen kurzen Pilot und danach die geplante kleine Studie durch. Dabei messe ich Bearbeitungszeit, Erfolgsquote und SUS, und nehme zusätzlich kurzes qualitatives Feedback mit, damit klar wird, welche Änderungen wirklich helfen und wo weiterhin Hürden bestehen.

8 Zeitplan

Woche	Phase	Ziele
1-2	Literatur & Rahmen	Aktuelle Quellen/WCAG 2.2/EU-Normen lesen, relevantes Kriterien-Set festlegen.
3	Szenarien & Vorbereitung	Szenarien definieren, Testprotokoll erstellen, Browser/Screenreader-Setup festlegen.
4–6	Evaluation der Plattformen	GitHub/GitLab/Azure anhand der Szenarien testen, Barrieren + positive Beispiele dokumentieren, vergleichen.
7	Schwerpunkte auswählen (für den Prototyp)	Wichtigste Probleme auswählen, Extension-Ziele und grobes Design festlegen.
8–10	Browser-Erweiterung bauen	Extension für 1–2 Workflows umsetzen, mit Tastatur/Screenreader testen und fixen.
11	Pilottest & Studienvorbereitung	Probelauf (1–2 Personen), Aufgaben/Ablauf schärfen, Rekrutierung + Consent vorbereiten.
12–13	Nutzerstudie & Auswertung	Studie ohne/mit Extension durchführen, Zeit/Erfolg/SUS auswerten.
14	Schreiben & Abgabe	Text fertigstellen, Diskussion/Limitationen, formaler Check.
15	Risikopuffer	Zeitreserve für Rekrutierung, UI-Änderungen, Technikprobleme.

9 Abbildungsverzeichnis

Abbildung 1: Github Issues-Übersicht. Markiert sind die Issues-Liste (links) und die Filter-Toolbar (rechts), in denen Barrieren bei Tab-/Fokus-Navigation beobachtet wurden..... 12

Abbildung 2: GitHub Issue-Detailansicht. Markiert ist die Markdown-Toolbar im Kommentar-Editor, deren Buttons sichtbar sind, aber nicht direkt per Tab erreichbar sind. 13

Abbildung 3: GitLab „Work items“ – Filter-/Suchbereich mit geöffnetem Token-Dropdown. Innerhalb des Widgets erfolgt die Auswahl typischerweise über Pfeiltasten und Enter, bevor die Tab-Navigation wieder fortgesetzt werden kann. 15

10 Quellenverzeichnis

- [1] World Wide Web Consortium, „Web Content Accessibility Guidelines (WCAG) 2.2“, World Wide Web Consortium, 2023. [Online]. Verfügbar unter: <https://www.w3.org/TR/WCAG22/>
- [2] S. Mealin und E. Murphy-Hill, „An exploratory study of blind software developers“, in *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, Innsbruck: IEEE, Sep. 2012, S. 71–74. doi: 10.1109/VLHCC.2012.6344485.
- [3] V. Potluri, P. Vaithilingam, S. Iyengar, Y. Vidya, M. Swaminathan, und G. Srinivasa, „CodeTalk: Improving Programming Environment Accessibility for Visually Impaired Developers“, in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, Montreal QC Canada: ACM, Apr. 2018, S. 1–11. doi: 10.1145/3173574.3174192.
- [4] M. Pandey, S. Bondre, S. O’Modhrain, und S. Oney, „Accessibility of UI Frameworks and Libraries for Programmers with Visual Impairments“, in *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, Roma, Italy: IEEE, Sep. 2022, S. 1–10. doi: 10.1109/VL/HCC53370.2022.9833098.
- [5] S. Nowrin und K. Vertanen, „Programming by Voice: Exploring User Preferences and Speaking Styles“, in *Proceedings of the 5th International Conference on Conversational User Interfaces*, Eindhoven Netherlands: ACM, Juli 2023, S. 1–13. doi: 10.1145/3571884.3597130.
- [6] P. T. Chiou, A. S. Alotaibi, und W. G. J. Halfond, „Detecting and localizing keyboard accessibility failures in web applications“, in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, Aug. 2021, S. 855–867. doi: 10.1145/3468264.3468581.
- [7] T. Bi, X. Xia, D. Lo, und A. Aleti, „A First Look at Accessibility Issues in Popular GitHub Projects“, in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Luxembourg: IEEE, Sep. 2021, S. 390–401. doi: 10.1109/ICSME52107.2021.00041.
- [8] European Parliament and Council of the European Union, „Directive (EU) 2016/2102 of the European Parliament and of the Council of 26 October 2016 on the accessibility of the websites and mobile applications of public sector bodies“, European Parliament and Council of the European Union, 2016/2102, 2016. [Online]. Verfügbar unter: <https://eur-lex.europa.eu/eli/dir/2016/2102/oj>
- [9] European Parliament and Council of the European Union, „Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the accessibility requirements for products and services“, European Union, 2019/882, 2019. [Online]. Verfügbar unter: <https://eur-lex.europa.eu/eli/dir/2019/882/oj>
- [10] „ETSI EN 301 549 - V3.2.1 - Accessibility requirements for ICT products and services“. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: https://www.etsi.org/deliver/etsi_en/301500_301599/301549/03.02.01_60/en_301549v030201p.pdf
- [11] „GitHub Accessibility Conformance Reports“, GitHub. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://accessibility.github.com>
- [12] „GitLab Accessibility Conformance Report International Edition | Pajamas Design System“. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://design.gitlab.com/accessibility/acr>
- [13] „Accessibility Conformance Reports | Microsoft Accessibility“, Microsoft. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://www.microsoft.com/en-us/accessibility/conformance-reports>

- [14] „Website Accessibility Conformance Evaluation Methodology (WCAG-EM) 1.0“. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/WCAG-EM/>
- [15] „Accessible Rich Internet Applications (WAI-ARIA) 1.2“. Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://www.w3.org/TR/wai-aria-1.2/>
- [16] W. W. A. Initiative (WAI), „ARIA Authoring Practices Guide“, Web Accessibility Initiative (WAI). Zugegriffen: 16. Januar 2026. [Online]. Verfügbar unter: <https://www.w3.org/WAI/ARIA/apg/>